The Effect of Poor Source Code Lexicon and Readability on Developer's Cognitive Load

Nikhil Anand

March 28, 2026

The paper investigates how the quality of source code lexicon and readability influences developers' cognitive load during program comprehension. Program comprehension is a fundamental activity in software development, and a significant portion of development effort is spent understanding existing code before making changes. The authors argue that identifiers (e.g., variable and method names) and comments serve as critical carriers of domain knowledge, and their quality directly affects how easily developers can interpret code. While prior work has established links between identifier quality and overall software quality, there remains limited empirical evidence on how poor lexicon specifically impacts developers' mental effort during comprehension tasks .

To address this gap, the study explores the relationship between linguistic and structural properties of code and the cognitive load experienced by developers. Using techniques such as functional Near Infrared Spectroscopy (fNIRS) and eye tracking, the authors measure cognitive effort at a fine-grained level while participants perform comprehension and bug localization tasks. The work focuses particularly on "linguistic antipatterns"—poor naming and documentation practices—and examines how these, along with readability issues, affect developers' ability to process code. Overall, the paper aims to provide empirical evidence that poor source code lexicon and readability increase cognitive load, thereby negatively impacting program comprehension and developer efficiency .

1 Motivation

A substantial portion of software development effort is devoted to understanding existing code before making modifications, making program comprehension a central and costly activity in the software lifecycle. Prior research highlights that identifiers and comments act as key vehicles for embedding domain knowledge within code, enabling developers to reason about functionality and intent. Empirical studies have shown that the quality of identifiers correlates with overall software quality, suggesting that poor naming and documentation practices may hinder comprehension and increase maintenance effort [1].

Despite these insights, there is limited empirical evidence directly linking the quality of source code lexicon—particularly naming inconsistencies and poorly chosen identifiers—to developers' cognitive load during comprehension tasks. While software engineering research has extensively explored readability and structural metrics, the cognitive processes underlying how developers interpret code remain underexplored. Advances in cognitive neuroscience and program comprehension research have introduced techniques such as fMRI and eye tracking to study developers' mental processes, yet these approaches are often expensive or lack granularity [2, 3]. This creates a clear need for empirical, fine-grained investigation into how linguistic aspects of code influence cognitive effort, motivating the study.

1.1 Background

The paper builds on the concept of linguistic antipatterns (LAs), which are recurring poor practices in naming, documentation, and identifier usage that negatively affect code understandability. These antipatterns include inconsistencies between method names and behavior, misleading identifiers, and contradictions between comments and implementation. Such issues can introduce confusion, forcing

developers to spend additional cognitive effort reconciling discrepancies between code and its intended meaning [4].

In addition to linguistic factors, prior work has examined structural and readability metrics—such as cyclomatic complexity, lines of code, nesting depth, and adherence to coding conventions—as indicators of code comprehensibility. Studies have shown that these metrics correlate with how easily developers can understand and maintain code, with poorly structured code increasing comprehension difficulty [5]. Furthermore, research in cognitive measurement has explored the use of physiological signals to study program comprehension. Techniques such as fMRI have been used to identify brain regions involved in code understanding, while eye tracking has provided insights into developers' visual attention patterns during reading tasks [2, 6].

More recently, functional Near Infrared Spectroscopy (fNIRS) has emerged as a practical alternative for measuring cognitive load, offering a non-invasive and portable way to capture brain activity during real-world programming tasks. Unlike traditional methods, fNIRS enables fine-grained analysis of cognitive effort while maintaining ecological validity, making it suitable for studying how specific code features—such as lexicon quality and readability—affect developers' mental workload.

1.1.1 Linguistic Antipatterns

The paper builds upon prior work on linguistic antipatterns (LAs), which capture recurring deficiencies in the way identifiers, comments, and method names convey meaning in source code. These antipatterns arise when there is a mismatch between the natural language used in code and the actual behavior or intent of the program. Examples include misleading method names, inconsistent terminology, or contradictions between comments and implementation. Such issues can obscure program intent and force developers to spend additional effort reconciling semantic inconsistencies. Prior research has demonstrated that developers rely heavily on identifiers and comments as primary cues for understanding code, making the quality of these linguistic elements crucial for comprehension [4].

Linguistic antipatterns are particularly problematic because they violate developers' expectations about naming conventions and semantic consistency. When identifiers fail to accurately reflect functionality, developers must rely more on low-level code analysis rather than high-level reasoning, increasing cognitive effort. Studies on naming quality and identifier consistency further support the idea that well-chosen names improve comprehension speed and accuracy, while poor naming introduces ambiguity and misunderstandings [1]. This body of work establishes the importance of lexicon quality as a key factor influencing program comprehension, motivating its deeper investigation in relation to cognitive load.

1.1.2 Cognitive Load and Program Comprehension

The second foundation of the paper lies in understanding cognitive load during program comprehension. Cognitive load refers to the mental effort required to process information, and in the context of software development, it reflects how difficult it is for a developer to understand and reason about code. Prior research in software engineering has explored the relationship between code complexity, readability, and comprehension, showing that factors such as structural complexity, nesting, and formatting significantly impact how easily developers can interpret code [5]. However, these studies often rely on indirect measures such as performance or subjective assessments, leaving a gap in directly quantifying cognitive effort.

To address this, recent work has incorporated physiological measurement techniques to study developers' mental processes. Approaches such as functional magnetic resonance imaging (fMRI) and eye tracking have been used to analyze how developers read and understand code, revealing insights into attention patterns and brain activity during comprehension tasks [2, 6]. More recently, functional Near Infrared Spectroscopy (fNIRS) has emerged as a practical alternative for measuring cognitive load in realistic settings, offering a balance between accuracy and usability. These advances provide the methodological foundation for studying how specific factors—such as poor lexicon and readability—affect developers' cognitive effort, enabling a more precise and empirical understanding of program comprehension.

2 Methodology

RQ 1. How do linguistic antipatterns affect developers' cognitive load during code comprehension tasks?

RQ 2. How does source code readability influence developers' cognitive load?

RQ 3. What is the combined effect of poor lexicon and low readability on cognitive load?

RQ 4. How do linguistic antipatterns and readability impact developers' task performance (e.g., comprehension and bug localization)?

2.1 Study Design

The study adopts a controlled experimental design to evaluate how linguistic antipatterns and readability influence cognitive load. Participants are asked to perform program comprehension and bug localization tasks on code snippets that are systematically manipulated to include or exclude linguistic antipatterns and readability issues. By controlling these variables, the study isolates their individual and combined effects on developer cognition.

The experiment follows a within-subject design, where each participant interacts with multiple code samples under different conditions. This allows for direct comparison of cognitive load across varying levels of lexicon quality and readability while minimizing variability due to individual differences.

2.2 Participants

The study involves participants with programming experience, typically including graduate students or developers with a background in software engineering. Participants are selected to ensure they possess sufficient familiarity with programming concepts required for comprehension tasks.

Before the experiment, participants are often screened or surveyed to assess their experience level, ensuring a relatively consistent baseline of programming proficiency. This helps reduce confounding factors related to expertise differences.

2.3 Experimental Tasks

Participants are assigned tasks that involve understanding code and identifying bugs. These tasks are designed to reflect realistic software engineering activities, particularly those requiring careful reasoning about program behavior.

The code snippets used in the tasks are modified to include linguistic antipatterns and varying levels of readability. This enables the study to evaluate how these factors influence both comprehension effort and task performance under controlled conditions.

2.4 Independent Variables

The primary independent variables in the study are the presence of linguistic antipatterns and the level of code readability. Linguistic antipatterns are introduced into code snippets to simulate poor naming and documentation practices. Readability is manipulated through structural characteristics such as formatting, complexity, and organization.

These variables are systematically combined to create different experimental conditions, allowing the study to analyze both individual and interaction effects on cognitive load.

2.5 Dependent Variables

The main dependent variable is cognitive load, which is measured using physiological signals obtained through fNIRS. This provides an objective measure of mental effort during task execution.

In addition to cognitive load, task performance metrics such as accuracy and completion time are recorded. These measures help assess how increased cognitive load translates into practical impacts on developer effectiveness.

2.6 Data Collection

Data is collected using a combination of physiological measurements and behavioral observations. fNIRS is used to capture brain activity associated with cognitive effort while participants perform tasks. This enables fine-grained tracking of mental workload in real time.

Alongside physiological data, the study records task outcomes such as correctness and time taken. This dual approach allows for a comprehensive analysis of both cognitive and performance-related effects.

2.7 Data Analysis

The collected data is analyzed to identify statistically significant differences across experimental conditions. Comparisons are made between code with and without linguistic antipatterns, as well as across different levels of readability.

The analysis focuses on understanding how these factors influence cognitive load and performance, both independently and in combination. This enables the study to draw conclusions about the role of lexicon quality and readability in program comprehension.

3 Key Findings

RQ1: Effect of Linguistic Antipatterns on Cognitive Load The results show that the presence of linguistic antipatterns significantly increases the cognitive load experienced by developers during program comprehension tasks. Participants exposed to code containing misleading or inconsistent identifiers exhibited higher mental effort, as measured through physiological signals. This indicates that poor naming forces developers to invest additional cognitive resources to interpret program intent. The study highlights that **linguistic inconsistencies disrupt semantic understanding**, leading to increased mental workload.

Furthermore, developers working with code containing linguistic antipatterns demonstrated reduced efficiency in reasoning about the code. Even when the structural complexity remained unchanged, the degraded lexicon quality alone contributed to higher cognitive strain. This suggests that **identifier quality independently impacts comprehension**, separate from structural code factors.

RQ2: Effect of Readability on Cognitive Load The findings indicate that reduced code readability also leads to a measurable increase in cognitive load. Code with poor formatting, higher complexity, or less clear structure required more effort for participants to process and understand. This reinforces the role of readability as a critical factor in supporting efficient program comprehension. The study emphasizes that **structural clarity directly reduces mental effort**, enabling developers to navigate and interpret code more effectively.

In addition, participants working with less readable code took longer to complete tasks and showed signs of increased cognitive strain. The lack of clear organization and formatting made it harder to follow control flow and logic. This demonstrates that **readability affects both cognitive load and task efficiency**, highlighting its importance in software quality.

RQ3: Combined Effect of Lexicon and Readability When linguistic antipatterns and poor readability were combined, the study observed the highest levels of cognitive load among participants. The interaction of semantic confusion (from poor naming) and structural complexity (from low readability) created compounding difficulties for developers. The results show that **the combined effect is greater than individual factors**, indicating a cumulative burden on cognitive processing.

Participants in these conditions not only experienced higher mental effort but also struggled more with understanding program behavior. The simultaneous presence of multiple issues reduced their ability

to form accurate mental models of the code. This suggests that **lexicon and readability jointly influence comprehension**, and improving only one may not be sufficient to reduce cognitive load effectively.

RQ4: Impact on Task Performance The study further demonstrates that increased cognitive load due to linguistic antipatterns and poor readability negatively affects task performance. Participants working with degraded code were more likely to make errors in comprehension and bug localization tasks. This highlights that cognitive strain translates directly into practical inefficiencies. The results indicate that **higher cognitive load leads to lower accuracy**, affecting developers' ability to correctly interpret code.

Additionally, task completion times were longer in conditions with poor lexicon and readability. Developers required more time to understand and reason about the code, reflecting increased effort and reduced productivity. This shows that **cognitive load impacts both correctness and speed**, reinforcing the importance of writing clear and well-structured code.

4 Threats to Validity

One potential threat to internal validity arises from the controlled experimental setting and the use of artificially constructed code snippets. While such control allows isolation of linguistic antipatterns and readability effects, it may not fully capture the complexity of real-world software systems. Participants may behave differently when working with larger, more familiar codebases compared to short experimental tasks. Additionally, the manipulation of code to inject antipatterns could introduce unintended artifacts that influence cognitive load beyond the intended variables. These factors suggest that **experimental control may limit realism**, potentially affecting the interpretation of causal relationships [2].

Threats to external validity stem from the participant pool and task design. The study primarily involves participants with academic or limited professional experience, which may not fully represent the diversity of expertise found in industry settings. Experienced developers might employ different strategies or be more resilient to poor naming and readability issues. Furthermore, the tasks focus on comprehension and bug localization, which, while common, do not cover the full spectrum of software engineering activities. This raises concerns that **results may not generalize across all developer populations and tasks** [6].

Construct validity is another concern, particularly in how cognitive load is measured and interpreted. Although fNIRS provides a promising physiological measure of mental effort, it captures brain activity indirectly and may be influenced by factors unrelated to the experimental variables, such as fatigue or individual differences in cognitive processing. Moreover, mapping physiological signals to cognitive load involves assumptions that may not fully reflect the nuanced mental processes involved in program comprehension. Therefore, **cognitive load measurements may not perfectly represent developers' true mental effort**, potentially affecting the accuracy of conclusions [2].

References

[1] Florian Deissenboeck and Markus Pizka. "Concise and Consistent Naming". In: *Proceedings of the 13th IEEE International Workshop on Program Comprehension*. 2006.

[2] Janet Siegmund et al. "Understanding Understanding Source Code with Functional Magnetic Resonance Imaging". In: *Proceedings of the 36th International Conference on Software Engineering*. 2014.

[3] Benjamin Floyd et al. "Decoding the Representation of Code in the Brain". In: *Proceedings of the 2017 IEEE/ACM International Conference on Software Engineering*. 2017.

[4] Venera Arnaoudova, Massimiliano Di Penta, and Giuliano Antoniol. "Linguistic Antipatterns: What They Are and How Developers Perceive Them". In: *Proceedings of the 2013 IEEE International Conference on Software Maintenance*. 2013.

[5] Raymond P. L. Buse and Westley Weimer. “Learning a Metric for Code Readability”. In: *Proceedings of the 2010 IEEE International Conference on Software Maintenance*. 2010.
[6] Thomas Fritz et al. “Using Psycho-Physiological Measures to Assess Task Difficulty in Software Development”. In: *Proceedings of the 36th International Conference on Software Engineering*. 2014.